When Does Design Thinking Work? A Convergence Theorem for the Iterate–Test Core

Paweł Kuraś[1], Adrian Michalski[2], Alicja Gerka[3], and Patryk Organiściak[*3]

[1]Institute of Telecommunications, AGH University of Science and Technology, Kraków, Poland
[2]Faculty of Mathematics and Applied Sciences, Rzeszów University of Technology, Rzeszów, Poland
[3]Department of Complex Systems, Rzeszów University of Technology, Rzeszów, Poland

Abstract

Design thinking is taught as a loop of ideation, prototyping and testing, and is credited with reliably improving what teams build, yet there is no account of the conditions under which such a loop must improve an outcome. We give one. Within a single framing of the problem we model the loop as a search over a finite space of prototypes whose utility is learned from noisy tests, and we prove that the loop converges almost surely to a local optimum of utility when five conditions hold: the frame is finite, ideation stays close to what has been built, the supply of improving ideas does not dry up too quickly, tests are calibrated and repeated, and nothing that has been tested is thrown away. Biased tests and slowly drifting criteria weaken the result by a bounded amount; with several stakeholders the limit is a locally Pareto-optimal set. Simulations on three classes of landscape show what the theorem leaves out. On landscapes of moderate ruggedness the limit is rarely reached: on our main landscape one run in twenty sits at a local optimum after 80 rounds and one in eight after 2,560. What limits a team is not finding good prototypes but recognising them. The favourite it would ship is persistently worse than the best prototype it has already tested, and its estimate of that favourite is inflated by about 8% of the optimum. Under a fixed testing budget, a team should build something new in every round and spend most of its tests re-testing its leading prototypes; testing every prototype once is the worst allocation we found. Confirmation bias is harmless while it is smaller than the noise in a test and costly beyond that. Given the same tests, the loop far outperforms random search and hill-climbing and comes within a point or two of a tuned bandit policy, and screening ideas roughly before building one is worth more than the choice between the two. Across framings, reframing once captures most of the available gain, and an expected-improvement rule for when to reframe does no better. Whether design teams meet the five conditions is an empirical question, and we describe the measurements that would answer it.

Keywords: design thinking; iteration; prototyping; Bayesian decision theory; convergence; rugged landscapes; exploration and exploitation

*org@prz.edu.pl

1 Introduction

Design thinking asks teams to understand the people they are designing for, frame a problem, generate many ideas, build cheap prototypes and test them with users, and to repeat the last three steps until something works (Brown, 2008; Hasso Plattner Institute of Design, 2010). The method has spread from product design into management, public policy and health care (Liedtka, 2015; Mintrom & Luetjens, 2016; Altman et al., 2018). It has done so without a theory of why it should work. Reviews of the literature describe a construct that is fragmented and thinly grounded (Johansson-Sköldberg et al., 2013; Micheli et al., 2019), and in health care, where the evidence has been reviewed systematically, it consists largely of small pilot studies (Altman et al., 2018). The method's own vocabulary does not help. The process is presented as a sequence of phases and also as a set of overlapping activities to be revisited as needed (Brown, 2008), which makes it hard to say what would count as doing it wrong.

This paper asks a narrow question that can be answered exactly. *Under what conditions does the iterate–test core of design thinking provably improve what a team builds, and what does the guarantee leave out?* We take the framing of the problem as given and study the loop that follows it: propose variants, build one, test it, update what is believed about every prototype built so far, and repeat. A prototype is a set of features, its value is a utility that the team does not know and can observe only through noisy tests, and the team keeps a belief about each prototype it has tested. In this setting "improvement" has a definite meaning, and so does "local optimum": a prototype that no nearby variant beats.

The first part of the paper is a convergence result. If the team entertains finitely many features, generates ideas near what it has already built, keeps producing improving ideas at a rate that is not summable, tests in a calibrated way and re-tests what it has kept, and never discards a tested prototype, then the loop converges with probability one, and what it converges to is a local optimum (Theorem 1). The argument is brief: the belief about the team's favourite is a bounded submartingale, which settles the convergence, and a conditional Borel–Cantelli argument identifies the limit. The interest lies in the conditions, each of which names a practice and each of which fails in a recognisable way. We also show how far the result bends: a bias of at most B in the tests leaves

the team at a prototype that no neighbour beats by more than $2B$; criteria that drift within a band of half-width D behave like a bias of size D; and when stakeholders disagree, the set of non-dominated prototypes converges to a locally Pareto-optimal set.

The second part asks what the theorem is worth in practice, and here the simulations change the story. Because the prototype space in our experiments is small enough to enumerate, we know exactly which prototypes are local optima and can check whether a run has reached one. On our main landscape it rarely has: one run in twenty holds a local optimum as its favourite after 80 rounds, and one in eight after 2,560. Only on very rugged landscapes, where local optima are many and close together, does a majority of runs get there, and then only after several hundred rounds. What a team gets at a realistic horizon is governed by rates, and three findings follow from looking at them.

First, recognition and not discovery is the bottleneck. After 80 rounds the best prototype a team has tested is worth 97.5% of the optimum, but the prototype it believes to be best is worth 94.8%, and the gap widens with time. The team also overrates its favourite, by about 8% of the optimum, which is the optimizer's curse of Smith & Winkler (2006) arising inside a design loop. Second, the division of a fixed testing budget between new prototypes and re-tests matters, and it matters in one direction. Spending every test on a new prototype is the worst allocation in every condition we tried; one to three new prototypes for every five tests, with the rest spent on the leading prototypes, is best, and the noisier the tests the more re-testing pays. Third, a confirmation bias in favour of the current favourite does no measurable harm while it is smaller than the noise of a test, and beyond that the loss grows quickly. The guarantee of the theory weakens in proportion to the bias; the typical loss has a threshold.

Two comparisons complete the picture. Given the same number of tests, the loop ships far better prototypes than random search or hill-climbing and stays within a point or two of an upper-confidence-bound policy tuned for the task, and screening ideas, however roughly, before one of them is built is worth more than the difference between the two. Across framings, a team that reframes once in mid-project does at least as well as one that follows an expected-improvement criterion for when to reframe, and both do better than a team that never reframes.

None of this shows that design teams satisfy the conditions. The model says what a team would have to do for iteration to be guaranteed to help; it does not claim that designers compute posteriors. Section 7 sets out what would have to be measured in real projects for the model to be tested and where it could turn out to be wrong.

2 Background and related work

The five modes of the Stanford d.school (empathize, define, ideate, prototype, test) (Hasso Plattner Institute of Design, 2010) and the Design Council's Double Diamond, which alternates divergent and convergent phases across discovery, definition, development and delivery (Design Council, 2007), are the two most widely taught descriptions. Brown's formulation adds the criteria by which an outcome is judged: what people want, what can be built, and what a business can sustain (Brown, 2008, 2009). Empirical studies of practice identify experimentation and iteration, alongside user focus and problem framing, as recurring elements of what organisations mean by the term (Carlgren et al., 2016; Micheli et al., 2019), and Liedtka (2015) argues that its value lies in countering predictable biases of judgement, among them the tendency to seek confirmation for a solution one already favours. Studies of novice teams find that outcomes depend on how need-finding, brainstorming and prototyping are combined and not merely on whether they occur (Seidel & Fixson, 2013). Our model takes the last three modes as its object and treats the first two as producing the frame within which they operate.

Formal theories of design begin with Simon, whose science of the artificial (Simon, 1996) casts design as search under bounded rationality, ending in a satisfactory alternative and not an optimal one (Simon, 1956). Later formal accounts describe what design reasoning operates on. The function–behaviour–structure ontology separates what an artefact is for, how it behaves and what it is (Gero & Kannengiesser, 2004); C–K theory describes design as the joint expansion of a space of concepts and a space of knowledge (Hatchuel & Weil, 2009); axiomatic design gives criteria for a good mapping from requirements to parameters (Suh, 1990); and models of design iteration identify which coupled tasks force rework (Smith & Eppinger, 1997). Co-evolution models hold that problem and solution develop together (Maher & Poon, 1996; Dorst & Cross, 2001), Schön (1983) places the setting of the problem at the centre of professional practice, and Dorst (2011) describes framing as an abductive step. These theories say what designers reason about. They do not state conditions under which an iterative process must improve its outcome, which is the gap we address. Our model is deliberately the simplest representation in which that question can be posed: design as exploration of a space of alternatives (Woodbury & Burrow, 2006), with the frame held fixed.

The convergence of a search process to a local optimum has well-known relatives outside design. Adaptive walks on rugged landscapes end at local optima after a number of steps that shrinks as ruggedness grows (Kauffman & Levin, 1987; Kauffman, 1993), and organisational search of this kind adapts well locally and misses distant peaks (Levinthal, 1997; Levinthal & March, 1993), the exploitation side of March's trade-off (March, 1991). A canonical genetic algorithm does not converge to the global optimum, while variants that always retain the best solution found do (Rudolph, 1994); retention plays a comparable role in our theorem. What distinguishes design from these settings is that a prototype's value is not observed but estimated. Search with fallible evaluation behaves differently from search with exact evaluation (Knudsen & Levinthal, 2007), and the economics of testing have been studied in their own right: testing as a sequence of experiments whose mode can be chosen (Thomke, 1998), the choice between testing alternatives in parallel and in sequence (Loch et al., 2001; Dahan & Mendelson, 2001),

and the way correlated designs shape a test sequence (Erat & Kavadias, 2008). Experimental work on ideation shows that the quality of the best idea depends on how well a group can tell good ideas from bad ones and not only on how many it generates (Girotra et al., 2010). Our results on recognition and on the allocation of tests belong to this line of work.

Finally, a team that maintains beliefs about alternatives and chooses what to test next is solving a sequential decision problem of the kind studied in Bayesian optimization (Jones et al., 1998; Shahriari et al., 2016), where expected improvement and the knowledge gradient (Frazier et al., 2008, 2009) are standard criteria and regret bounds are available for smooth objectives (Srinivas et al., 2012). In a finite space with independent beliefs the closer analogy is best-arm identification, where the number of tests needed to separate two alternatives grows with the inverse square of the difference between them (Even-Dar et al., 2006; Mannor & Tsitsiklis, 2004; Audibert et al., 2010). We use these results in two ways: to explain why recognition is slow, and to supply the baselines against which the loop is compared (Auer et al., 2002; Russo et al., 2018).

3 The model

The model has two levels, the choice of a frame and the loop that runs inside it, and three ingredients: prototypes, a utility, and tests.

3.1 Frames, prototypes and the loop

A project moves through the five modes of design thinking, and it often moves backwards: a test that reveals a misread need sends the team back to its users, and one that reveals a weak concept sends it back to ideation (Figure 1). We split this movement into two levels. The outer level is the choice of a *frame*. Empathizing gathers observations about users, and defining commits the team to a point of view, a statement of whose problem is being solved and what would count as solving it. In Dorst's account this step is abductive: from what it has observed, the team conjectures both the kind of thing to be made and the reason it would be valuable (Dorst, 2011; Kolko, 2010). We do not model how the conjecture is produced. For our purposes its result is a pair (F, U): a finite set F of features that solutions may combine, and a utility U that says how good each combination is. A *reframe* replaces (F, U) by another pair, and a *framing epoch* is the stretch of the project between two reframes.

The inner level is the loop inside an epoch. A *prototype* is a subset $p \subseteq F$, so the space of prototypes is 2^F. In round k the team does three things. It *ideates*: it proposes variants of prototypes it has already built, which is the divergent step. It *prototypes*: it chooses among its untested ideas and builds one. It *tests*: it gathers evidence about the new prototype and about some of the older ones and revises its beliefs, and in doing so it decides which prototype is currently its favourite, which is the convergent step. A small example fixes ideas. A team designing a travel mug has settled on the commuter who drinks while walking, and its frame admits four features: a folding handle, a one-touch lid, a spill seal and a temperature display. There are sixteen prototypes, from the bare mug to the mug with everything. If ideation changes one feature at a time, the variants of the mug with a seal are the bare mug and the three mugs that add one further feature to the seal. The mug with a lid and a seal is a local optimum if adding the handle or the display, or removing the lid or the seal, makes it worse on balance, even if the mug with a handle and a display and no lid, three changes away, would be better still. Deciding that the commuter is the wrong user is a reframe.

We write $P_k \subseteq 2^F$ for the set of prototypes built and tested by the end of round k and $\mathcal{F}_k$ for everything the team has observed by then. Table 1 collects the notation.

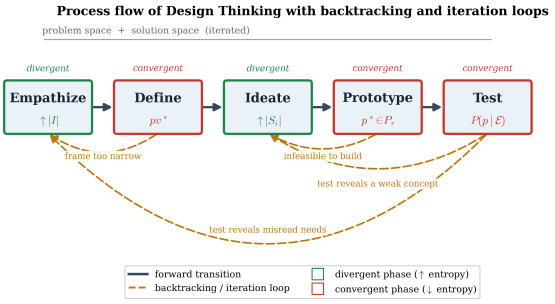


Figure 1: The design thinking process as modelled. Solid arrows are the forward sequence; dashed arrows are returns. The theorem concerns the loop among Ideate, Prototype and Test within one framing; a return to Empathize or Define starts a new framing epoch.

Table 1: Notation.

F, 2^F	features admitted by the frame; prototype space
$U(p)$	utility of prototype p, in $[0,1]$; $U^* = \max_p U(p)$
$\mathcal{N}(p)$	variants of p that ideation can reach in one step
P_k	prototypes built and tested by round k
$\mathcal{F}_k$	evidence available after round k
M_k^p	belief about p: the posterior mean $\mathbb{E}[U(p) \mid \mathcal{F}_k]$
p_k^*, X_k	the favourite, $\arg\max_{p \in P_k} M_k^p$, and its belief
δ_k	lower bound on the chance that round k builds an improving variant of the favourite
σ, B	standard deviation and maximal bias of a test

3.2 Utility

Following Brown (2009), a solution is judged by its desirability to users, its technical feasibility and its business viability. In our examples the utility is a weighted sum of the three,

$$U(p) = w_d U_D(p) + w_v U_V(p) + w_f U_F(p), \qquad (1)$$

with each component in $[0,1]$ and weights summing to one, as in multi-attribute utility theory (Keeney & Raiffa, 1976). Nothing in the theory depends on this form. The results use only two properties of U: it is bounded, and it does not change during an epoch. A utility in which a failure on one criterion cannot be compensated by the others satisfies both. Section 4.4 drops the assumption that stakeholders agree on a single U.

3.3 Tests and beliefs

The team does not know U. A test of prototype p in round k returns

$$y = U(p) + b_k + \varepsilon_k, \tag{2}$$

where the noise terms ε_k are independent of each other, of U and of the team's choices, with mean zero and variance σ^2, and b_k is a systematic error with $|b_k| \le B$. A user interview, an expert critique and a feasibility estimate are all tests in this sense, and they differ in σ and in how prone they are to bias. The team treats U as uncertain: it starts from a prior distribution over utilities and updates it with each result in the usual Bayesian way (DeGroot, 2004). Probability statements below refer to this prior together with the randomness of tests and of ideation, and "almost surely" is meant in that joint sense. Its belief about p after round k is summarised by the posterior mean $M_k^p = \mathbb{E}[U(p) \mid \mathcal{F}_k]$. Its *favourite* is the tested prototype with the highest belief, and

$$X_k = \max_{p \in P_k} M_k^p \tag{3}$$

is how good the team thinks its favourite is. Three other quantities will matter and should be kept apart from X_k: the true utility $U(p_k^*)$ of the favourite, which is what the team would actually deliver if it stopped now and which we call the *shipped* utility; the best true utility among all tested prototypes, $\max_{p \in P_k} U(p)$, which is what it has found whether or not it knows it; and the utility of the best favourite so far, $Y_k = \max_{j \le k} U(p_j^*)$.

Beliefs exist only for prototypes that have been tested. The choice of which untested idea to build next must therefore rest on something cheaper, such as a rough estimate, an analogy or a critique. We call this *prescreening*. It affects how fast the loop moves and not where it ends.

4 Convergence within a frame

This section states the conditions, proves the result, and examines what remains of it when the conditions are weakened.

4.1 Five conditions

Assumption 1 (Finite frame)**.** Within an epoch the feature set F is finite and the utility $U : 2^F \to [0,1]$ does not change. Almost surely, no two prototypes have exactly the same utility.

The exclusion of ties is a convenience that holds generically when criteria are measured on a continuous scale. Finiteness is a real restriction, because it rules out the arrival of features nobody had thought of. If features are added during an epoch but the additions stop, the results apply from the last addition onward; if they never stop, there is no fixed space in which to speak of an optimum.

Assumption 2 (Local ideation)**.** Each prototype p has a set $\mathcal{N}(p) \subseteq 2^F$ of variants, and every new idea is a variant of a prototype already in P_k. A prototype is *locally optimal* if no member of $\mathcal{N}(p)$ has higher utility.

In the simulations $\mathcal{N}(p)$ consists of the prototypes that differ from p in at most two features. Locality is what makes the limit a local optimum. If $\mathcal{N}(p)$ were the whole space, the same argument would give convergence to the global optimum, at a price in speed that Section 5.5 measures.

Assumption 3 (Persistent generativity)**.** There are numbers $\delta_k \ge 0$, determined by the history up to round $k-1$ and by U, such that, whenever the favourite p_{k-1}^* has a better variant that has not yet been built, round k builds and tests such a variant with conditional probability at least δ_k, and

$$\sum_{k \ge 1} \delta_k = \infty \quad \text{almost surely.} \tag{4}$$

The condition allows ideas to become harder to find as the team closes in on a peak. A chance of improvement that falls like $1/k$ satisfies (4); one that falls like $1/k^2$ does not. The constant case $\delta_k \ge \delta > 0$ is the special case in which returns do not diminish at all. The assumption concerns building only. The team need not recognise the better variant when it builds it; recognition is the business of the next assumption.

Assumption 4 (Calibrated, repeated testing)**.** Tests are unbiased ($B = 0$), the team's beliefs are the posterior means of a correctly specified model, and every prototype in P_k continues to be tested, so that each is tested infinitely often.

Two consequences are used below. Because $M_k^p = \mathbb{E}[U(p) \mid \mathcal{F}_k]$ is the conditional expectation of one bounded random variable given a growing body of evidence, it is a martingale and converges almost surely to $\mathbb{E}[U(p) \mid \mathcal{F}_\infty]$. And because the average of infinitely many independent tests of p converges to $U(p)$ by the law of large numbers, $U(p)$ is determined by the evidence in the limit, so that $M_k^p \to U(p)$: beliefs become correct.

Assumption 5 (Retention)**.** No tested prototype is discarded: $P_k \subseteq P_{k+1}$.

4.2 The theorem

Theorem 1. *Under Assumptions 1, 4 and 5, the belief X_k about the favourite is a bounded submartingale and converges almost surely and in L^1. If Assumptions 2 and 3 also hold then, almost surely, the favourite is eventually a single prototype p_∞, this prototype is locally optimal, and $X_k \to U(p_\infty)$.*

Proof. For fixed p, M_k^p is a martingale with values in $[0,1]$. Retention gives $P_k \subseteq P_{k+1}$, and a maximum of martingales is a submartingale by the conditional Jensen inequality, so

$$\begin{aligned} \mathbb{E}[X_{k+1} \mid \mathcal{F}_k] &\ge \mathbb{E}\Big[\max_{p \in P_k} M_{k+1}^p \,\Big|\, \mathcal{F}_k\Big] \\ &\ge \max_{p \in P_k} \mathbb{E}[M_{k+1}^p \mid \mathcal{F}_k] = X_k. \end{aligned}$$

Since $0 \le X_k \le 1$, the martingale convergence theorem gives a limit almost surely (Durrett, 2019, Sec. 4.2), and by bounded convergence also in L^1.

For the second part, P_k is a non-decreasing sequence of subsets of the finite set 2^F, so there is a random round k_0 after which it equals a fixed set P_∞. Every $p \in P_\infty$ is tested infinitely often, hence $M_k^p \to U(p)$. The utilities of the finitely many members of P_∞ are distinct, so from some round on the favourite is the member p_∞ with the highest utility, and $X_k \to U(p_\infty)$.

Suppose that, with positive probability, p_∞ has a variant $q \in \mathcal{N}(p_\infty)$ with $U(q) > U(p_\infty)$. Such a q cannot belong to P_∞, since p_∞ is the best member. On this event the hypothesis of Assumption 3 holds in every late round, so the conditional probabilities of building a better unbuilt variant sum to infinity. By Lévy's extension of the Borel–Cantelli lemma (Durrett, 2019, Sec. 4.3), applied in the filtration generated by the history together with U, such a variant is then built infinitely often, which contradicts $P_k = P_\infty$ for $k \geq k_0$. Hence p_∞ is locally optimal almost surely. □

Each assumption does one job in the proof, and removing it breaks a specific step. Retention makes the set over which the maximum is taken grow, which gives the submartingale. Calibration makes each belief a martingale and repetition makes it correct in the limit; without repetition a lucky first test could keep a mediocre prototype on top for ever. Generativity is what prevents the process from stopping short of a local optimum. Locality determines which optimum is meant.

The first part of the theorem is a statement about beliefs that use all the evidence under a correctly specified model. A team whose prior is much more diffuse than the real spread of utilities, or which ignores part of what it knows, holds beliefs of a different kind. Its estimate of its favourite overshoots, because the favourite is chosen for having the highest estimate, and is corrected only by further tests. This is the optimizer's curse (Smith & Winkler, 2006), and Section 5.2 shows that it is large. The second part of the theorem survives such misspecification, since it uses only the fact that beliefs become correct.

4.3 Biased tests and drifting criteria

Real tests are not unbiased. People seek and read evidence in ways that favour what they already believe (Nickerson, 1998), and a team that has a favourite concept is no exception. Suppose the bias in (2) is bounded by B and the team is unaware of it, so that it updates as if tests were unbiased. Write $\widehat{M}_k^p$ for the resulting beliefs.

Proposition 1. *Let Assumptions 1, 2 and 5 hold, let every retained prototype be tested infinitely often with $|b_k| \leq B$, let beliefs be formed by the conjugate normal update, and let Assumption 3 hold with $\delta_k \geq \delta$ for a constant $\delta > 0$. Then almost surely, from some round on, every favourite is $2B$-locally optimal: none of its variants has a utility exceeding its own by more than $2B$. For $B = 0$ this is the local optimality asserted in Theorem 1.*

Proof. The conjugate posterior mean is a weighted average of the prior mean and the test results, with weight on the prior tending to zero. Shifting each result by at most B shifts the average by at most B, so $\limsup_k |\widehat{M}_k^p - U(p)| \leq B$ for every $p \in P_\infty$. Let u be the highest utility in P_∞. A member with utility below $u - 2B$ has beliefs eventually below those of the best member, so every late favourite has utility at least $u - 2B$. Let H_k be the event that the favourite at round $k-1$ has a better variant not yet built. Each time H_k occurs a new prototype is built with probability at least δ; if H_k occurred infinitely often, infinitely many prototypes would be built, by the same Borel–Cantelli argument, which is impossible in a finite space. So eventually no favourite has a better unbuilt variant, and every variant that has been built has utility at most u, which exceeds the favourite's by at most $2B$. □

The bound is a worst case. It allows the bias to be arranged as unfavourably as possible, always against the better of two close prototypes. A bias that merely flatters the favourite matters only when it is large enough to overturn a comparison, and the simulated loss in Section 5.5 is accordingly flat for small biases and steep for large ones.

The same argument covers criteria that move. Suppose that during an epoch the utility is U_k at round k, for instance because the weights in (1) are renegotiated, and that it stays within a band around some reference utility $\bar{U}$ that has no ties.

Corollary 1. *If $|U_k(p) - \bar{U}(p)| \leq D$ for all k and p, then under the conditions of Proposition 1, with unbiased tests of the current utility and with "better variant" in Assumption 3 understood with respect to $\bar{U}$, every late favourite is $2D$-locally optimal for $\bar{U}$ and $4D$-locally optimal for every U_k.*

Proof. A test of $U_k(p)$ is a test of $\bar{U}(p)$ with a bias of at most D, so Proposition 1 applies with $B = D$. For a late favourite p and a variant q, $U_k(q) - U_k(p) \leq \bar{U}(q) - \bar{U}(p) + 2D \leq 4D$. □

The corollary does not cover criteria that drift without bound. A team whose notion of a good outcome keeps moving must discount old evidence, and a loop that discounts evidence is a different process from the one analysed here. A discontinuous change of criteria is a reframe.

4.4 Several stakeholders

A single utility presumes that the weights in (1) have been agreed. When they have not, let each stakeholder s in a finite set $\mathcal{S}$ have a utility $U^{(s)}$, say that q *dominates* p if $U^{(s)}(q) \geq U^{(s)}(p)$ for every s with strict inequality for some s, and let each test return one noisy observation per stakeholder. The team now keeps, in place of a favourite, an *archive* A_k: the tested prototypes that no other tested prototype dominates according to current beliefs.

Proposition 2. *Let Assumptions 1, 2, 4 and 5 hold for every stakeholder, with no ties in any $U^{(s)}$. Suppose that whenever a member of the archive has a dominating variant that has not been built, round k builds such a variant with conditional probability at least δ_k, where the δ_k are as in Assumption 3 and $\sum_k \delta_k = \infty$. Then almost surely the archive is eventually constant, it is the set of non-dominated members of P_∞, and none of its members has a dominating variant in $\mathcal{N}$.*

Proof. As before $P_k = P_\infty$ from some round on and all beliefs converge to the true utilities. With no ties, dominance according to beliefs eventually agrees with true dominance on the finite set P_∞, so the archive settles on the non-dominated members of P_∞. If a member p had a dominating variant q, then $q \notin P_\infty$, because otherwise p would not be in the archive. The hypothesis of the generativity condition would then hold in every late round and a dominating variant would be built, contradicting $P_k = P_\infty$. □

With one stakeholder the archive is the favourite and the proposition is Theorem 1. With several, the loop does not resolve the disagreement; it delivers a set of mutually incomparable prototypes, each of which is locally unimprovable for everyone at once, and leaves the choice among them to negotiation. This is the behaviour of archive-based evolutionary multi-objective methods (Laumanns et al., 2002), and the proposition shows that it needs nothing beyond the conditions of the scalar case.

4.5 How fast?

Almost-sure convergence says nothing about time. Two observations bound the time from different sides.

The first concerns discovery. Suppose for a moment that tests are exact, so that the favourite is always the best prototype built so far, and that $\delta_k \geq \delta > 0$. Let T be the first round at which the favourite is locally optimal and Λ the number of times the favourite improves up to and including that round.

Proposition 3. *With exact tests and $\delta_k \geq \delta$, $\mathbb{E}[T] \leq \mathbb{E}[\Lambda]/\delta$.*

Proof. Let N_k be the number of improvements by round k. In each round before T an improvement occurs with conditional probability at least δ, so $N_{k\wedge T} - \delta\,(k \wedge T)$ is a submartingale and $\delta\,\mathbb{E}[k \wedge T] \leq \mathbb{E}[N_{k\wedge T}] \leq \mathbb{E}[\Lambda]$. Let $k \to \infty$. □

Λ is at most the number of distinct utility levels, and when ideation always starts from the favourite it is at most the length L of the longest chain of successively better variants. On the main landscape of Section 5, $L = 532$ for variants within two features, so the bound allows thousands of rounds even before noise is considered.

The second concerns recognition. With noisy tests the team must also come to believe that a better prototype is better. Telling apart two prototypes whose utilities differ by Δ requires a number of tests of order σ^2/Δ^2 on each (Even-Dar et al., 2006; Mannor & Tsitsiklis, 2004). Near the top of a landscape the differences are small. With $\sigma = 0.1$ and prototypes that differ by one hundredth of a utility unit, a single comparison needs hundreds of tests. The quadratic dependence on Δ is the reason the last comparisons in a project are the most expensive ones, and it is why recognition lags behind discovery in the simulations.

4.6 Stopping and reframing

Because the limit is local, more iteration cannot repair a poor frame. A team therefore faces two decisions that the theorem does not address: when to stop, and when to reframe. Both can be put on one scale. The value of another round within the frame is the *expected improvement*

$$\mathrm{EI}_k = \max_p \mathbb{E}\big[(U(p) - X_k)^+ \mid \mathcal{F}_k\big], \tag{5}$$

the criterion of Bayesian optimization (Jones et al., 1998) and a close relative of the knowledge gradient, which values a test by what it is expected to add to the best belief (Frazier et al., 2008). If a round costs c_{iter} in the same units, iterating is worthwhile while $\mathrm{EI}_k > c_{\mathrm{iter}}$. Read in this way, Simon's aspiration level is a price: cheap tests justify long iteration, expensive ones an early stop (Simon, 1956).

A reframe costs c_{ref}, the effort of going back to users and rebuilding context, and offers an uncertain gain G_{ref} over what the team already holds. A one-step comparison suggests reframing when

$$\mathrm{EI}_k - c_{\mathrm{iter}} < G_{\mathrm{ref}} - c_{\mathrm{ref}}. \tag{6}$$

The left side is computed from the team's own beliefs. The right side contains the one quantity that cannot be derived from inside the current frame. It can be estimated as an expected improvement one level up: if the outcomes that the team attains in different frames are regarded as draws from a normal distribution with mean m and standard deviation s, learned from earlier projects and updated after each epoch, then

$$G_{\mathrm{ref}} = (m - Y_k)\,\Phi(z) + s\,\phi(z), \qquad z = \frac{m - Y_k}{s}, \tag{7}$$

with Φ and ϕ the standard normal distribution and density functions and Y_k replaced in practice by the team's estimate of what it holds. Frames are then the arms of a bandit, iteration is exploitation and reframing is exploration in the sense of March (1991), and (6) is the greedy policy for that bandit. It is a heuristic and not an optimal rule; optimal search over alternatives with known reward distributions has the reservation-price structure of Weitzman (1979), and choosing between trial-and-error within a frame and parallel pursuit of several is treated by Sommer & Loch (2004). Section 5.7 compares it with simpler rules.

5 Simulation study

The simulations implement the loop of Section 3, so their agreement with the theorem would show only that the code is correct. Their use is to measure what the theorem does not say: how far a team gets in a realistic number of rounds, which of the conditions matter most at that horizon, and how the loop compares with other ways of spending the same tests. All code, seeds and raw outputs are in the repository listed at the end of the paper.

5.1 Setup

A prototype is a subset of $|F| = 14$ features, so there are $2^{14} = 16{,}384$ prototypes. This is small enough to compute the utility of every one, and therefore to know the optimum U^* and the complete set of local optima exactly.

We use three classes of landscape. The *main landscape* is an instance of (1) with weights $(0.45, 0.30, 0.25)$: desirability rewards the coverage of latent user needs with diminishing returns, viability rewards business value and penalises scope, and feasibility falls with build cost and with the cost of integrating pairs of features. Its optimum is $U^* = 0.763$, its mean utility 0.670 and the standard deviation of utility across prototypes 0.046; it has four local optima when variants differ in one feature and two when they may differ in two. The *DVF family* consists of 30 landscapes of the same form with independently drawn needs, values, costs and interactions (one to six local optima for one-feature variants, median three). The *NK family* consists of 20 landscapes from Kauffman's NK model (Kauffman, 1993), in which each feature's contribution depends on a number of other features, here 1, 2, 4 or 6, with five landscapes for each. These are far more rugged, with between 1 and 185 local optima for one-feature variants (median 38), and they share nothing with the main landscape beyond the size of the space. A result that holds on all three classes does not depend on how we chose to write down desirability, viability and feasibility.

A run starts from five random ideas. In each round the team picks one of its three *leading prototypes*, those with the highest beliefs, at random and proposes five variants of it, each differing from it in one or two features. It builds the untested idea that looks best on a rough estimate and tests it once. It then re-tests its three leading prototypes, and finally re-tests one older prototype, taking the older prototypes in rotation so that every one of them keeps being tested, as Assumption 4 requires. A round therefore uses five tests. Tests follow (2) with $\sigma = 0.10$, about twice the spread of utilities across the main landscape, and no bias. Beliefs are normal with a deliberately uninformative prior (mean 0.5, variance 0.08) and are confined to $[0, 1]$. The rough estimate used for pre-screening is the true utility plus noise of standard deviation 0.10, drawn once per idea; it is as informative as one test and costs nothing, an advantage that Sections 5.5 and 5.6 remove.

Utilities are reported as a percentage of the landscape's optimum U^*, and differences in *points* are differences in that percentage. The main outcome is the shipped utility $U(p_k^*)$, the true utility of the favourite. We also report the best utility *found*, $\max_{p \in P_k} U(p)$, and the team's belief X_k. Intervals are 95% confidence intervals; on the two families they are computed across landscapes from landscape means.

5.2 What a team has after 80 rounds

Figure 2 shows the baseline on the main landscape. Progress is fast for about twenty rounds and slow afterwards. After 80 rounds (400 tests) the team has found a prototype worth 97.5% of the optimum and would ship one worth 94.8%. The difference of 2.7 points is the cost of not knowing which of its prototypes is best, and it is larger than everything the favourite gained between round 20 and round 80 (1.5 points).

The team's belief about its favourite tells a different story from the favourite's utility. It passes 100% of the optimum at round 18 and ends at 102.4%: the team believes it holds something better than the best prototype that exists. The overestimate, 0.058 ± 0.004 in utility units or 7.6% of the optimum, is the optimizer's curse (Smith & Winkler, 2006). The favourite is chosen for having the highest estimate, and a high estimate is partly luck; an uninformative prior does nothing to discount the luck. Replacing the prior by one matched to the landscape (mean 0.670, standard deviation 0.046) cuts the overestimate to 0.014 ± 0.002 and leaves the shipped utility unchanged (94.6% against 94.8%). Calibration corrects what the team believes about its result. It does not improve the result.

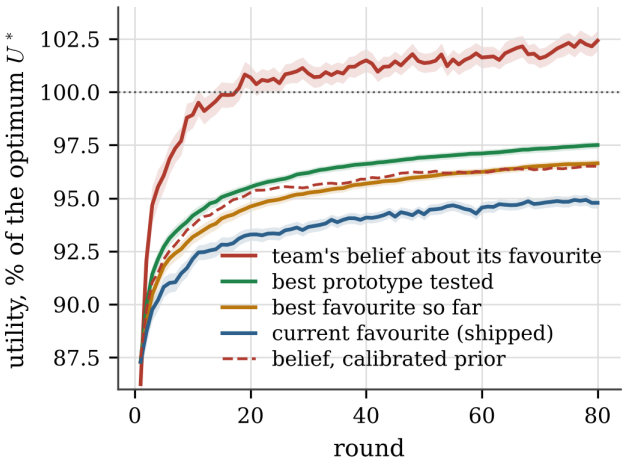


Figure 2: The baseline loop on the main landscape (250 runs; bands are 95% confidence intervals). The team's belief about its favourite exceeds the optimum from round 18 on, while the favourite itself is worth less than the best prototype the team has tested. A prior calibrated to the landscape (dashed) removes most of the overestimate.

5.3 The limit is rarely reached

To see where the process is heading we ran it for up to 2,560 rounds and checked at each horizon whether the favourite is a local optimum, by looking it up in the enumerated set (Figure 3). On the main landscape it is one in 5% of runs after 80 rounds, still in 5% after 640, and in 13% after 2,560 rounds, which is 12,800 tests in a space of 16,384 prototypes. Restricting variants to a single feature raises the last figure to 19%. On the DVF family the share is 2% after 80 rounds and 15% after 1,280. On the rugged NK landscapes, where local optima are numerous and close together, it is 26% after 80 rounds and 72% after 1,280.

On landscapes of moderate ruggedness, then, a team that stops after a few dozen rounds has not converged, and the value of its result is set by how quickly the loop climbs and not by where it would end. A run that finishes below the global optimum should not be read as having settled at a local one: on the main landscape nearly all runs finish below the global optimum and very few at a local optimum.

The dashed curves in Figure 3 show that discovery is not what holds the process back. After 2,560 rounds on the main landscape the best prototype tested is a local optimum in 88% of runs and is worth 99.9% of the

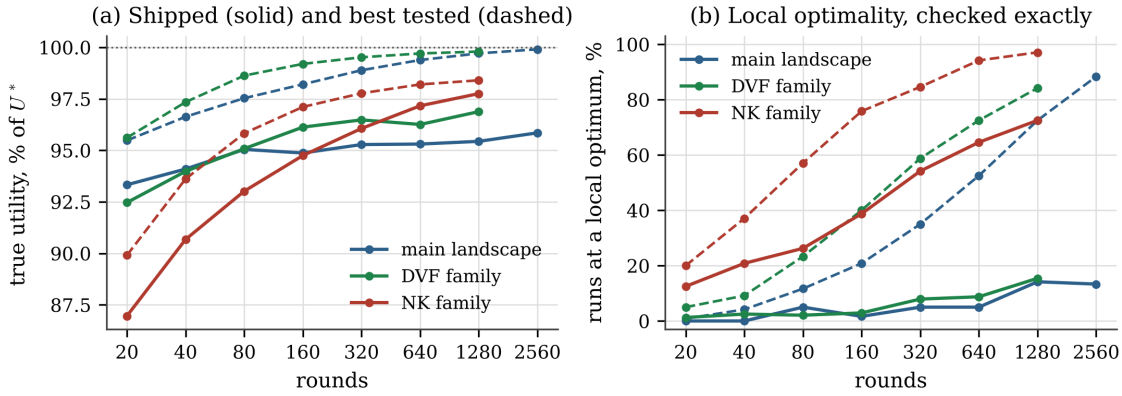


Figure 3: Long runs (main landscape: 120 runs; DVF and NK families: 10 landscapes each, 24 runs per landscape). **(a)** Utility shipped (solid) and best utility among tested prototypes (dashed). **(b)** Share of runs in which the favourite (solid) or the best tested prototype (dashed) is a local optimum, determined exactly by enumeration.

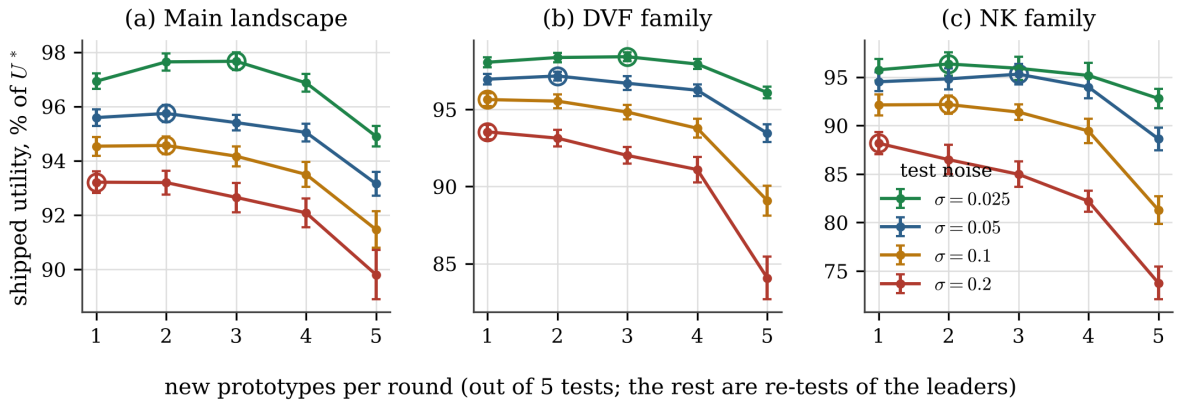


Figure 4: Shipped utility after 60 rounds of five tests each, as a function of how many of the five tests go to new prototypes, for four levels of test noise (five ideas per round; error bars are 95% confidence intervals; circles mark the best allocation). Spending all five tests on new prototypes is the worst allocation at every noise level.

optimum, while the favourite is worth 95.8%. The team has found a near-optimal prototype and does not know it. The reason is the quadratic cost of comparison noted in Section 4.5. Late in a run the team holds thousands of prototypes within a few percent of each other, most of them tested only a handful of times, and the highest of several thousand noisy estimates rarely belongs to the best prototype. One remedy is to discount thin evidence. A team that ships the prototype with the highest belief minus one posterior standard deviation, in place of the highest belief, gains little at 80 rounds (95.4% against 95.0%) but 2.4 points at 2,560 (98.2% against 95.8%) on the main landscape, and 1.2 points at 1,280 rounds on the DVF family.

5.4 New prototypes or re-tests?

If recognition is the bottleneck, the division of tests between new prototypes and old ones should matter. We fixed the budget at five tests per round for 60 rounds and varied how many of the five went to new prototypes, from one to all five, with the remainder spent re-testing the leading prototypes (Figure 4; 120 runs on the main landscape and 16 on each landscape of the families).

The clearest result is at the right-hand end of each curve. Giving all five tests to new prototypes, so that nothing is ever tested twice, is the worst allocation in all 24 combinations of landscape class, noise level and ideation volume that we ran. Against the best allocation it costs about 3 points on the main landscape, 2 to 10 on the DVF family and 4 to 14 on the NK family, and the cost grows with the noise. Four new prototypes and a single re-test is the second worst allocation nearly everywhere. Between one and three new prototypes per round the curves are flat by comparison: the differences are mostly within the confidence intervals, and the best allocation is never more than three. There is a tendency, visible in Figure 4, for the best allocation to move towards re-testing as tests get noisier, from two or three new prototypes at $\sigma = 0.025$ to one at $\sigma = 0.20$. Tripling the number of ideas generated per round, from 5 to 15, moves the curves by about half a point on the main landscape and the DVF family and by one to two points on the NK family, and does not change their shape.

A team should build something new in every round and spend at least half of its testing effort confirming what it already has, more when its tests are unreliable. Building many prototypes and testing each once is the pattern to avoid.

Table 2: One change at a time: utility shipped after 80 rounds, as a percentage of U^*, with 95% confidence half-widths (main landscape: 200 runs; families: 16 runs on each landscape). The baseline uses variants within two features, five ideas per round, full retention, three re-tests of leading prototypes and one rotating re-test per round, and $\sigma = 0.10$. The row without re-testing uses one test per round where the others use five, and its ideas are variants of the favourite alone; Section 5.4 gives the comparison at equal budget.

Condition	Changed element	Main	DVF family	NK family
Baseline	—	94.9 ± 0.3	95.7 ± 0.5	93.7 ± 0.9
Locality	variants within 1 feature	94.5 ± 0.3	95.2 ± 0.4	92.5 ± 1.1
	variants within 3 features	94.7 ± 0.3	95.9 ± 0.4	93.3 ± 0.8
	unbounded variants	94.2 ± 0.3	94.9 ± 0.5	92.4 ± 0.8
Generativity	1 idea per round	94.5 ± 0.3	94.4 ± 0.5	89.6 ± 1.2
	2 ideas per round	94.8 ± 0.3	95.2 ± 0.4	92.0 ± 0.9
	15 ideas per round	94.8 ± 0.3	95.8 ± 0.4	94.0 ± 0.9
Retention	only the favourite kept	93.7 ± 0.3	93.8 ± 0.6	90.1 ± 1.0
	best 3 kept	94.9 ± 0.2	95.7 ± 0.4	94.0 ± 0.9
	best 10 kept	94.9 ± 0.3	96.0 ± 0.4	93.4 ± 0.9
Testing	no re-testing	92.5 ± 0.4	91.6 ± 0.6	84.3 ± 1.2
	no rotating re-test	94.7 ± 0.3	95.7 ± 0.4	93.0 ± 0.9
	noise $\sigma = 0.05$	95.9 ± 0.2	97.5 ± 0.4	95.4 ± 1.0
	noise $\sigma = 0.20$	93.5 ± 0.3	93.6 ± 0.5	88.6 ± 1.3
Pre-screening	ideas built in random order	94.0 ± 0.3	94.3 ± 0.6	89.0 ± 1.4

5.5 Relaxing the conditions

Table 2 changes one element of the baseline at a time, grouped by the assumption each change bears on.

The reach of ideation matters little within limits. Narrowing it to one-feature variants costs 0.4 points on the main landscape and the DVF family and 1.2 on the NK family, and widening it to three features changes nothing measurable. Removing the bound altogether, so that a variant may differ from its parent in any number of features, lowers the result by 0.7 to 1.3 points. In the limit unbounded ideation reaches the global optimum, but within 80 rounds it wastes tests on distant prototypes that are mostly poor.

The supply of ideas matters on rugged landscapes and saturates quickly elsewhere. A team that produces one idea per round ships 0.4 points less than the baseline on the main landscape and 4.0 points less on the NK family, and fifteen ideas per round are no better than five on any class. Which idea is built matters more than how many there are to choose from: building a random idea, in place of the one that looks best on the rough estimate, costs 0.9, 1.4 and 4.7 points on the three classes.

Retention matters in a specific way. A team that remembers only its current favourite loses 1.2 to 3.6 points, and remembering three prototypes recovers all of it. The practical content of the retention condition is therefore that the favourite must have rivals to be compared with, not that everything must be archived.

Testing matters most. Dropping all re-tests is the most damaging change in the table, at 2.3, 4.1 and 9.4 points. Dropping only the rotating re-test of older prototypes, which the theorem needs for beliefs to become correct, costs at most 0.6 points at this horizon. Halving the noise of a test is worth 1.0 to 1.8 points and doubling it costs 1.4 to 5.1.

Figure 5 adds a confirmation bias: every re-test of the current favourite is inflated by b. At the baseline noise of 0.10 no loss is detectable on any class up to $b = 0.10$; at $b = 0.20$ the loss is 1.0 to 3.5 points and at $b = 0.30$ it is 3.7 to 8.0. The point at which losses begin moves with the noise. With more precise tests ($\sigma = 0.05$) losses are visible from $b = 0.075$ and reach 1.1 to 2.5 points at $b = 0.10$; with noisier tests ($\sigma = 0.20$) nothing is lost up to $b = 0.20$. A bias protects the favourite against challengers, and it changes the outcome only when it is large enough to reverse comparisons that the evidence would otherwise have decided, which here means roughly as large as the standard deviation of a test. Precise tests make a team more vulnerable to its own bias, not less, because there is less noise for a challenger to be lucky in. Proposition 1 gives a guarantee that weakens in proportion to the bias; the typical loss has a threshold.

Taken together, the table and the figure identify testing as the condition that matters most at realistic horizons, in both its aspects: how reliable a test is, and whether leading prototypes are tested more than once. The quality of ideation comes second and matters mainly where the landscape is rugged.

5.6 Comparison with other policies

Is the loop a good way to spend a testing budget, or merely a convergent one? We compared it with five alternatives given the same 300 tests, the same variants, the same test noise and the same belief update (Table 3): random search among variants; hill-climbing that tests each one-feature variant of its current point several times and moves to the best; an ε-greedy policy that with probability ε tests a new variant of the favourite and otherwise re-tests the favourite; an upper-confidence-bound (UCB) policy (Auer et al., 2002); and Thompson sampling (Russo et al., 2018), the last two adding a new variant

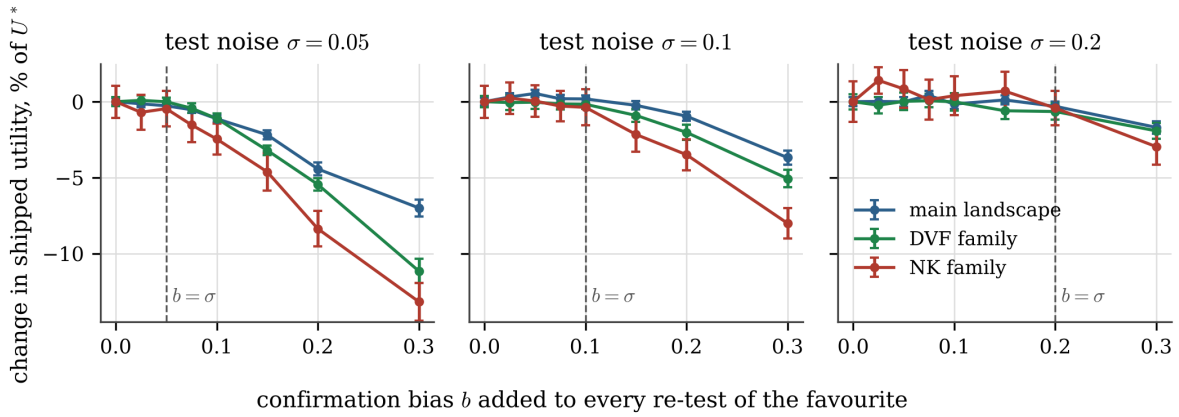


Figure 5: Change in shipped utility after 80 rounds when every re-test of the favourite is inflated by b, at three levels of test noise (main landscape: 240 runs per point; families: 16 runs per landscape; error bars are 95% confidence intervals). The dashed line marks $b = \sigma$. Losses are negligible while the bias is smaller than the noise and grow quickly beyond it.

Table 3: Utility shipped after 300 tests, as a percentage of U^*, on landscapes not used for tuning (main landscape: 250 runs; families: 20 landscapes each, 30 runs per landscape; $\pm$ is the 95% confidence half-width). "Diff." is the mean paired difference from the loop across landscapes.

	Main	DVF family		NK family	
Policy	shipped	shipped	diff.	shipped	diff.
Random search	91.0 ± 0.4	87.9 ± 0.8	-7.4 ± 0.8	80.2 ± 0.9	-12.0 ± 0.9
Hill-climbing	91.0 ± 0.5	88.7 ± 0.8	-6.6 ± 0.8	83.8 ± 1.4	-8.4 ± 1.6
ε-greedy	94.0 ± 0.3	94.3 ± 0.6	-1.1 ± 0.3	89.9 ± 1.1	-2.3 ± 0.6
Upper confidence bound (UCB)	94.5 ± 0.3	94.7 ± 0.5	-0.6 ± 0.3	90.2 ± 1.1	-2.1 ± 0.5
Thompson sampling	93.9 ± 0.3	94.4 ± 0.7	-0.9 ± 0.4	89.6 ± 1.1	-2.7 ± 0.6
UCB with pre-screening	95.0 ± 0.2	96.1 ± 0.4	$+0.8 \pm 0.2$	93.6 ± 1.0	$+1.4 \pm 0.5$
Loop without pre-screening	93.8 ± 0.3	93.9 ± 0.6	-1.5 ± 0.5	87.5 ± 1.2	-4.7 ± 0.7
Loop	94.7 ± 0.3	95.3 ± 0.4	—	92.2 ± 0.9	—

of the favourite at regular intervals. Because the loop screens its ideas before building one and these policies do not, we also ran the loop without pre-screening, and the UCB policy with it, in the form of the best of five fresh variants on the same rough estimate. The parameters of each alternative were tuned on ten DVF and eight NK landscapes, and all policies were then scored on landscapes and seeds not used for tuning. The loop was run with its baseline settings.

The table separates two things that an informal comparison would confound. The first is the policy for spending tests. Random search and hill-climbing are 4 to 12 points behind the loop: a policy that does not concentrate its tests on promising prototypes wastes most of them. Among the policies that do concentrate their tests, the differences are much smaller, and the loop is not the best of them. Without pre-screening it is level with ε-greedy and Thompson sampling on the main landscape and half a point to one point behind UCB on the main landscape and the DVF family; on the NK family it is two to three points behind all three. With pre-screening on both sides, UCB is ahead of the loop by 0.3 points on the main landscape, which is within noise, and by 0.8 and 1.4 points on the families, where it wins on nine landscapes in ten.

The second is pre-screening. It is worth 0.9, 1.5 and 4.7 points to the loop and 0.5, 1.4 and 3.5 points to UCB, which is more than the difference between the two policies on every class.

The loop is therefore a good way to spend a testing budget and not the best one. A fixed routine of one new prototype and four re-tests per round comes within a point or two of an index policy tuned for the purpose, and far ahead of search that does not re-test. What matters more than the choice between them is that ideas are screened before one is built, even by an estimate as noisy as a single test.

5.7 Reframing

The last experiment concerns the outer level. A world contains six frames, each a different DVF landscape drawn from a pool of 40. A team has 150 rounds. It works in one frame at a time with the loop of Section 5.1, here without the rotating re-test, and a reframe costs c_{ref} rounds and moves it to a frame it has not visited, where it starts again. The payoff is the utility of the best favourite it held in any frame. We compare a team that never reframes, teams that reframe on a fixed schedule every κ rounds, and teams that follow criterion (6), with a cost of 0.002 per round and with G_{ref} computed from (7) under three priors for the outcome of a fresh frame: a default (mean 0.75, standard deviation 0.05), one calibrated to the outcomes

Table 4: Best utility attained within 150 rounds across six frames, by reframing policy and cost of a reframe in rounds (240 paired runs; 95% half-widths are at most 0.004). The last column (refr.) is the mean number of reframes per run at a cost of 10 rounds; the three criterion rows differ in the prior used for G_{ref}. Bold marks the best policy at each cost.

	cost of a reframe (rounds)				
Policy	4	10	20	35	refr.
Never	0.709	0.709	0.709	0.709	0
Every 10 rounds	0.719	0.720	0.715	0.706	5.0
Every 15 rounds	0.722	0.719	0.716	0.708	5.0
Every 25 rounds	**0.725**	**0.721**	0.711	0.715	5.0
Every 40 rounds	0.723	0.720	0.717	0.714	3.0
Once, at round 75	0.720	0.720	**0.719**	**0.717**	1.0
Criterion, default	0.716	0.716	0.717	0.715	0.6
Criterion, calibrated	0.717	0.717	0.714	0.712	0.3
Criterion, optimistic	0.718	0.716	0.717	0.716	0.7

actually attained in these frames (0.708 and 0.028), and an optimistic one (0.758 and 0.028). The criterion is given the true utility of the team's best favourite, which favours it. Every policy sees the same 240 worlds at each of four reframing costs.

A single reframe at the midpoint beats never reframing at every cost, by 0.008 to 0.011 in utility, a little over one percent of a typical optimum (Table 4). How often to reframe depends on the cost. When a reframe is cheap, reframing every 25 rounds is best, and when it is expensive, the single reframe is; reframing every 10 or 15 rounds is never best and is worse than never reframing when a reframe costs 35 rounds. The criterion beats never reframing at every cost, by 0.006 to 0.008, but it does not beat the simple schedules. It triggers only 0.2 to 0.7 reframes per run. It is slightly worse than the single scheduled reframe when reframing is cheap (-0.005 ± 0.002 at a cost of 4 rounds) and indistinguishable from it when reframing is expensive. A calibrated prior makes it more cautious still and no better.

In this model, then, most of the value lies in reframing at all, and a rule as crude as "go back to the problem once, halfway through" captures it. Criterion (6) is useful as a statement of what the decision depends on. As a rule to follow it is too conservative, because it compares a reframe with one more round of iteration, and one more round of iteration almost always looks worth having.

6 Discussion

We first answer the question in the title, then consider what follows for tools and where the model stops.

6.1 When design thinking works

The question in the title has an answer at two levels. In the limit, the iterate–test loop works when the five conditions hold, and the conditions translate into practices that the design thinking literature already recommends for other reasons. Keep what you have tried, so that the best of it remains available. Test in a way that can contradict you, and test your favourites again. Keep producing variants of what you have, even when they are becoming harder to find. Stay within reach of what you have built, and accept that what you will get is the best thing in the neighbourhood, not the best thing possible. When these hold, improvement is not a matter of luck or talent; it follows from the structure of the process. This is a precise version of Liedtka's argument that the method works by disciplining judgement (Liedtka, 2015).

At the horizon of an actual project the answer is different, and less comfortable for the way the method is usually taught. Design thinking is usually presented as a discipline of generating: many ideas, many cheap prototypes, an early move from talking to building (Brown, 2008; Hasso Plattner Institute of Design, 2010). In our simulations generation is not scarce. Teams find good prototypes early and go on to find excellent ones. What they lack is the ability to tell which of their prototypes is the good one. The favourite a team would ship is worse than the best thing it has tested at every horizon, its confidence in that favourite is inflated, and the most valuable use of a test is to repeat one. The pattern that an emphasis on building invites, many different prototypes each shown to users once, is the worst allocation we found. The finding agrees with experimental evidence that the best idea a group ends up with depends on how well it discriminates among its ideas (Girotra et al., 2010), and with the observation that imperfect evaluation changes the character of search (Knudsen & Levinthal, 2007).

Four recommendations follow. A team should screen its ideas before building, since a rough estimate of which idea is most promising was worth more in our comparisons than the difference between a good testing policy and a slightly better one. It should treat its estimate of its favourite concept as an overestimate; the optimizer's curse grows with the number of alternatives chosen among (Smith & Winkler, 2006). It should spend most of its testing on the two or three leading concepts and prefer a well-tested concept to one that looked excellent once. And it should worry about confirmation bias most when its tests are precise: a bias smaller than the noise of a test changed nothing in our simulations, and a bias larger than the noise decided the outcome.

The result on reframing has the same character. Iteration cannot escape a poor frame, and in the simulations a team that never reframes does worse than one that reframes once. A rule that watches the expected improvement of further iteration added nothing to a fixed plan, which makes the practical advice simple: plan for at least one return to the problem definition, and do not expect iteration to signal when it is due.

Table 5 summarises the conditions, what each asks of a team, and what its violation cost in the simulations.

6.2 Implications for tools

Shared whiteboards and similar tools help design teams generate and arrange ideas. The model suggests that the scarcer resource is evaluation, and it specifies what a tool that supported evaluation would do: keep a belief and an evidence count for every concept tested, shrink estimates

Table 5: The five conditions of Theorem 1: what each means in practice, what the theory says when it fails, and the measured cost of violating it after 80 rounds (range over the three landscape classes, in points of U^*; from Table 2 and Figure 5).

Condition	In practice	When it fails	Cost in simulation
Finite frame	The brief and the vocabulary of features stay fixed during an epoch	No fixed space in which an optimum is defined	Not simulated
Local ideation	New ideas are variants of what has been built	With unbounded jumps the limit is global but progress is slower	Unbounded variants: -0.7 to -1.3
Persistent generativity	The team keeps proposing variants, even late	The process can stop short of a local optimum	One idea per round: -0.4 to -4.0
Calibrated, repeated testing	Tests can contradict the team; favourites are tested again	A lucky prototype can stay on top; under bias B the result is only $2B$-locally optimal	No re-testing: -2.3 to -9.4. Bias $b = 0.2$: -1.0 to -3.5
Retention	Rivals to the favourite are kept	The best prototype found can be lost	Only the favourite kept: -1.2 to -3.6

that rest on little evidence, show the team how much of its favourite's lead is likely to be selection, and propose the next test by its expected improvement. The same reasoning applies to generative AI as a source of ideas. A language model raises the number of ideas per round at almost no cost and may raise their individual quality, while reducing their diversity (Doshi & Hauser, 2024). In the model, more ideas per round help only on rugged landscapes, and less diverse ideas mean a smaller effective neighbourhood. Where such a model is more useful is in pre-screening, if it can rank ideas even roughly before any is built. Cheap ideation by itself moves the bottleneck further towards testing.

6.3 Limitations

The model is normative. It describes a team that keeps beliefs and updates them consistently, and real teams do so informally at best. Its predictions concern what happens as a team's practice comes closer to or moves further from the conditions, not what any given team does.

The frame is taken as given. The model has nothing to say about how a point of view is formed or what makes one frame better than another, which is where much of the skill in design lies and where problems are wicked in the sense of Rittel & Webber (1973) and Buchanan (1992): without a definitive formulation or a stopping rule. The reframing experiment treats frames as interchangeable draws from a population, which is the least that can be assumed about them.

Within the frame, three simplifications are worth naming. Beliefs about different prototypes are independent, whereas a real test of one concept teaches something about its relatives; correlated beliefs would speed up recognition and would call for the correlated form of the knowledge gradient (Frazier et al., 2009). The feature set is fixed, which excludes the discovery of a new kind of feature in mid-epoch. And utility is a single number or a vector of numbers, which presumes that stakeholders can evaluate a prototype consistently from one test to the next.

The simulations use synthetic landscapes with fourteen features. The qualitative results hold across three classes of landscape of very different ruggedness, but the magnitudes should not be read as estimates for real projects. The threshold for bias was located at three noise levels only, and the rough pre-screening of ideas, which costs nothing in our simulations, would cost something in practice.

7 Testing the model on real teams

The results so far are statements about a model. This section describes what would have to be observed in real projects to test it.

7.1 Predictions

Five consequences of the model can be checked against records of design projects.

The first is a *recognition gap*: if all concepts a team tested are later rated by independent judges, the concept the team carried forward should often not be the highest rated, and the shortfall should be larger for teams that tested each concept fewer times. The second is *overconfidence*: a team's own assessment of its chosen concept should exceed the independent rating, by more when it chose among more candidates. The third concerns *allocation*: across teams with similar testing effort, outcomes should be best at an intermediate share of tests spent on new concepts and should fall off sharply as that share approaches one. The fourth is a *bias threshold*: leading questions and tests confined to the favourite should be associated with worse outcomes only where the bias they introduce is large relative to the variability of test results, and so more often in teams whose tests are precise. The fifth concerns *reframing*: teams that return to the problem definition at least once should do better than teams that never do.

If teams turned out to carry forward their best-rated concept nearly always, the recognition problem that dominates our simulations would be an artefact of the model's independent beliefs. If outcomes rose steadily with the number of distinct prototypes built, the allocation result would be wrong where it matters.

7.2 Measurement

The quantities involved can be obtained from recorded design sessions by protocol analysis, which has a long history in design research (Cross et al., 1996). A session is segmented into moves, and each move is labelled as proposing a concept, building one, testing one, or returning to the problem definition. For each test the coder records which concept it concerned and the direction and strength of the evidence on an ordinal scale, together with markers of likely bias such as a leading question or a test of the favourite alone. Two coders should label an overlapping sample and report agreement, for which Krippendorff's α is the usual measure (Krippendorff, 2018). That such coding supports quantitative analysis of sequences is established: Sung & Kelley (2019), for example, applied sequential analysis to the think-aloud protocols of young pupils and identified statistically reliable pathways among design activities, with idea generation the dominant one.

The generativity condition can be examined in the same records. For each round one counts whether a concept better than the current favourite, by the later independent ratings, was built; the share of such rounds among those in which a better variant demonstrably existed estimates δ_k, and its course over an epoch shows whether returns to ideation diminish and how fast. Summability itself cannot be tested on a finite record, but a rate that collapses to zero within an epoch would count against the condition.

Two design decisions are essential. The unit of analysis is the framing epoch, not the project, because the model's claims hold within a frame. And the outcome must be measured independently of the team's own beliefs, by judges who rate concepts without knowing which one the team chose. A study that reconstructed the team's beliefs from the coded tests and then asked whether those beliefs rose would test the coding scheme against itself.

7.3 An illustration and its limits

To see whether the coding is feasible on published material we applied it to one detailed case. van Asselt & Roke (2025) report the development of a stress-monitoring application for autistic adults through the stages of design thinking, with repeated refinement in collaboration with users. From their account we coded 14 test events, converted each to a value in $[0, 1]$, and reconstructed the belief about the leading concept. The reconstructed belief rose over the project by 0.015, with two downward steps, and the one extension for which testing showed no benefit over the application alone, a link to a wearable device, was not pursued.

The exercise shows that a published case can be coded and no more than that. The case is a reported success, the coding is ours and was done by a single coder, and the result is fragile: when each coded outcome is moved one step up or down the scale with probability 0.3, to imitate a second coder who disagrees, the rise in belief survives in only 61% of 20,000 re-codings. A test of the predictions above requires many epochs, including unsuccessful ones, two coders, and independent outcome ratings. We regard such a study as the necessary next step and have not carried it out.

8 Conclusion

Within a fixed framing of a problem, the loop of ideation, prototyping and testing converges to a locally optimal prototype provided that the frame is finite, ideation is local, improving ideas keep coming at a non-summable rate, tests are calibrated and repeated, and tested prototypes are retained. The guarantee degrades gradually under bounded bias and bounded drift of the criteria, and it extends to several stakeholders as convergence to a locally Pareto-optimal set.

The simulations show that this limit describes where the process is going and not where a team will be when it stops. Except on very rugged landscapes, few runs reach a local optimum at realistic horizons, and the quality of the result depends on how well the team can recognise the best of what it has built. A prototype tested once is a poor basis for a decision, the team's confidence in its favourite is systematically too high, confirmation bias matters once it exceeds the noise in the tests, and one reframe captures most of what reframing has to offer. A plain routine of one new prototype and four re-tests per round comes within a point or two of a bandit policy tuned for the task, and rough screening of ideas before building is worth more than the difference between them.

Design thinking works, in the sense made precise here, when a team keeps its options, generates near them, and above all tests them more than once. Whether teams that do these things obtain better outcomes than teams that do not is a question for field data, and the model says which data would settle it.

Data and code availability

The code that produces every figure, table and number in this paper, together with the raw outputs, is available at `https://github.com/pawelkuras/design-thinking-formalization` and will be deposited in a permanent archive on acceptance. The experiments use fixed random seeds and require only `numpy` and `matplotlib`; `run_all.sh` reproduces them from scratch.

CRediT authorship contribution statement

Paweł Kuraś: Conceptualization, Methodology, Formal analysis, Software, Writing – original draft. **Adrian Michalski:** Formal analysis, Validation, Writing – review & editing. **Alicja Gerka:** Conceptualization, Investigation, Writing – review & editing. **Patryk Organiściak:** Formal analysis, Validation, Writing – review & editing.

Declaration of competing interest

The authors declare no competing financial interests or personal relationships that could have influenced the work reported in this paper.

Declaration of generative AI use

A large-language-model assistant (Claude, Anthropic) was used extensively in preparing this manuscript: to draft and revise the text, to write and run the simulation code, to propose and carry out the simulation studies of Section 5, to restate and check the proofs, and to verify the bibliography against publishers' records. The authors defined the research question and the model, directed and reviewed this work, and are responsible for the content of the paper.

References

Altman, M., Huang, T. T. K., & Breland, J. Y. (2018). Design thinking in health care. *Preventing Chronic Disease, 15*, E117. doi:10.5888/pcd15.180128

Audibert, J.-Y., Bubeck, S., & Munos, R. (2010). Best arm identification in multi-armed bandits. In A. T. Kalai & M. Mohri (Eds.), *Proceedings of the 23rd Conference on Learning Theory (COLT 2010)* (pp. 41–53). Omnipress.

Auer, P., Cesa-Bianchi, N., & Fischer, P. (2002). Finite-time analysis of the multiarmed bandit problem. *Machine Learning, 47*(2–3), 235–256. doi:10.1023/A:1013689704352

Brown, T. (2008). Design thinking. *Harvard Business Review, 86*(6), 84–92.

Brown, T. (2009). *Change by Design: How Design Thinking Transforms Organizations and Inspires Innovation*. HarperBusiness.

Buchanan, R. (1992). Wicked problems in design thinking. *Design Issues, 8*(2), 5–21. doi:10.2307/1511637

Carlgren, L., Rauth, I., & Elmquist, M. (2016). Framing design thinking: The concept in idea and enactment. *Creativity and Innovation Management, 25*(1), 38–57. doi:10.1111/caim.12153

Cross, N., Christiaans, H., & Dorst, K. (Eds.). (1996). *Analysing Design Activity*. Wiley.

Dahan, E., & Mendelson, H. (2001). An extreme-value model of concept testing. *Management Science, 47*(1), 102–116. doi:10.1287/mnsc.47.1.102.10666

DeGroot, M. H. (2004). *Optimal Statistical Decisions*. Wiley-Interscience. (Original work published 1970)

Design Council. (2007). *Eleven Lessons: Managing Design in Eleven Global Brands. A Study of the Design Process*. Design Council.

Dorst, K. (2011). The core of 'design thinking' and its application. *Design Studies, 32*(6), 521–532. doi:10.1016/j.destud.2011.07.006

Dorst, K., & Cross, N. (2001). Creativity in the design process: Co-evolution of problem–solution. *Design Studies, 22*(5), 425–437. doi:10.1016/S0142-694X(01)00009-6

Doshi, A. R., & Hauser, O. P. (2024). Generative AI enhances individual creativity but reduces the collective diversity of novel content. *Science Advances, 10*(28), eadn5290. doi:10.1126/sciadv.adn5290

Durrett, R. (2019). *Probability: Theory and Examples* (5th ed.). Cambridge University Press. doi:10.1017/9781108591034

Erat, S., & Kavadias, S. (2008). Sequential testing of product designs: Implications for learning. *Management Science, 54*(5), 956–968. doi:10.1287/mnsc.1070.0784

Even-Dar, E., Mannor, S., & Mansour, Y. (2006). Action elimination and stopping conditions for the multi-armed bandit and reinforcement learning problems. *Journal of Machine Learning Research, 7*, 1079–1105.

Frazier, P. I., Powell, W. B., & Dayanik, S. (2008). A knowledge-gradient policy for sequential information collection. *SIAM Journal on Control and Optimization, 47*(5), 2410–2439. doi:10.1137/070693424

Frazier, P., Powell, W., & Dayanik, S. (2009). The knowledge-gradient policy for correlated normal beliefs. *INFORMS Journal on Computing, 21*(4), 599–613. doi:10.1287/ijoc.1080.0314

Gero, J. S., & Kannengiesser, U. (2004). The situated function–behaviour–structure framework. *Design Studies, 25*(4), 373–391. doi:10.1016/j.destud.2003.10.010

Girotra, K., Terwiesch, C., & Ulrich, K. T. (2010). Idea generation and the quality of the best idea. *Management Science, 56*(4), 591–605. doi:10.1287/mnsc.1090.1144

Hasso Plattner Institute of Design at Stanford. (2010). *An Introduction to Design Thinking: Process Guide*. Stanford University, d.school.

Hatchuel, A., & Weil, B. (2009). C–K design theory: An advanced formulation. *Research in Engineering Design, 19*(4), 181–192. doi:10.1007/s00163-008-0043-4

Johansson-Sköldberg, U., Woodilla, J., & Çetinkaya, M. (2013). Design thinking: Past, present and possible futures. *Creativity and Innovation Management, 22*(2), 121–146. doi:10.1111/caim.12023

Jones, D. R., Schonlau, M., & Welch, W. J. (1998). Efficient global optimization of expensive black-box functions. *Journal of Global Optimization, 13*(4), 455–492. doi:10.1023/A:1008306431147

Kauffman, S. A. (1993). *The Origins of Order: Self-Organization and Selection in Evolution*. Oxford University Press.

Kauffman, S., & Levin, S. (1987). Towards a general theory of adaptive walks on rugged landscapes. *Journal of Theoretical Biology, 128*(1), 11–45. doi:10.1016/S0022-5193(87)80029-2

Keeney, R. L., & Raiffa, H. (1976). *Decisions with Multiple Objectives: Preferences and Value Tradeoffs*. Wiley.

Knudsen, T., & Levinthal, D. A. (2007). Two faces of search: Alternative generation and alternative evaluation. *Organization Science, 18*(1), 39–54. doi:10.1287/orsc.1060.0216

Kolko, J. (2010). Abductive thinking and sensemaking: The drivers of design synthesis. *Design Issues, 26*(1), 15–28. doi:10.1162/desi.2010.26.1.15

Krippendorff, K. (2018). *Content Analysis: An Introduction to Its Methodology* (4th ed.). SAGE.

Laumanns, M., Thiele, L., Deb, K., & Zitzler, E. (2002). Combining convergence and diversity in evolutionary multiobjective optimization. *Evolutionary Computation, 10*(3), 263–282. doi:10.1162/106365602760234108

Levinthal, D. A. (1997). Adaptation on rugged landscapes. *Management Science, 43*(7), 934–950. doi:10.1287/mnsc.43.7.934

Levinthal, D. A., & March, J. G. (1993). The myopia of learning. *Strategic Management Journal, 14*(S2), 95–112. doi:10.1002/smj.4250141009

Liedtka, J. (2015). Perspective: Linking design thinking with innovation outcomes through cognitive bias reduction. *Journal of Product Innovation Management, 32*(6), 925–938. doi:10.1111/jpim.12163

Loch, C. H., Terwiesch, C., & Thomke, S. (2001). Parallel and sequential testing of design alternatives. *Management Science, 47*(5), 663–678. doi:10.1287/mnsc.47.5.663.10480

Maher, M. L., & Poon, J. (1996). Modeling design exploration as co-evolution. *Microcomputers in Civil Engineering, 11*(3), 195–209. doi:10.1111/j.1467-8667.1996.tb00323.x

Mannor, S., & Tsitsiklis, J. N. (2004). The sample complexity of exploration in the multi-armed bandit problem. *Journal of Machine Learning Research, 5*, 623–648.

March, J. G. (1991). Exploration and exploitation in organizational learning. *Organization Science, 2*(1), 71–87. doi:10.1287/orsc.2.1.71

Micheli, P., Wilner, S. J. S., Bhatti, S. H., Mura, M., & Beverland, M. B. (2019). Doing design thinking: Conceptual review, synthesis, and research agenda. *Journal of Product Innovation Management, 36*(2), 124–148. doi:10.1111/jpim.12466

Mintrom, M., & Luetjens, J. (2016). Design thinking in policymaking processes: Opportunities and challenges. *Australian Journal of Public Administration, 75*(3), 391–402. doi:10.1111/1467-8500.12211

Nickerson, R. S. (1998). Confirmation bias: A ubiquitous phenomenon in many guises. *Review of General Psychology, 2*(2), 175–220. doi:10.1037/1089-2680.2.2.175

Rittel, H. W. J., & Webber, M. M. (1973). Dilemmas in a general theory of planning. *Policy Sciences, 4*(2), 155–169. doi:10.1007/BF01405730

Rudolph, G. (1994). Convergence analysis of canonical genetic algorithms. *IEEE Transactions on Neural Networks, 5*(1), 96–101. doi:10.1109/72.265964

Russo, D. J., Van Roy, B., Kazerouni, A., Osband, I., & Wen, Z. (2018). A tutorial on Thompson sampling. *Foundations and Trends in Machine Learning, 11*(1), 1–96. doi:10.1561/2200000070

Schön, D. A. (1983). *The Reflective Practitioner: How Professionals Think in Action*. Basic Books.

Seidel, V. P., & Fixson, S. K. (2013). Adopting design thinking in novice multidisciplinary teams: The application and limits of design methods and reflexive practices. *Journal of Product Innovation Management, 30*(S1), 19–33. doi:10.1111/jpim.12061

Shahriari, B., Swersky, K., Wang, Z., Adams, R. P., & de Freitas, N. (2016). Taking the human out of the loop: A review of Bayesian optimization. *Proceedings of the IEEE, 104*(1), 148–175. doi:10.1109/JPROC.2015.2494218

Simon, H. A. (1956). Rational choice and the structure of the environment. *Psychological Review, 63*(2), 129–138. doi:10.1037/h0042769

Simon, H. A. (1996). *The Sciences of the Artificial* (3rd ed.). MIT Press.

Smith, R. P., & Eppinger, S. D. (1997). Identifying controlling features of engineering design iteration. *Management Science, 43*(3), 276–293. doi:10.1287/mnsc.43.3.276

Smith, J. E., & Winkler, R. L. (2006). The optimizer's curse: Skepticism and postdecision surprise in decision analysis. *Management Science, 52*(3), 311–322. doi:10.1287/mnsc.1050.0451

Sommer, S. C., & Loch, C. H. (2004). Selectionism and learning in projects with complexity and unforeseeable uncertainty. *Management Science, 50*(10), 1334–1347. doi:10.1287/mnsc.1040.0274

Srinivas, N., Krause, A., Kakade, S. M., & Seeger, M. W. (2012). Information-theoretic regret bounds for Gaussian process optimization in the bandit setting. *IEEE Transactions on Information Theory, 58*(5), 3250–3265. doi:10.1109/TIT.2011.2182033

Suh, N. P. (1990). *The Principles of Design*. Oxford University Press.

Sung, E., & Kelley, T. R. (2019). Identifying design process patterns: A sequential analysis study of design thinking. *International Journal of Technology and Design Education, 29*(2), 283–302. doi:10.1007/s10798-018-9448-1

Thomke, S. H. (1998). Managing experimentation in the design of new products. *Management Science, 44*(6), 743–762. doi:10.1287/mnsc.44.6.743

van Asselt, A., & Roke, Y. (2025). Co-creating the Stress Autism Mate (SAM): Development of an mHealth application for autistic adults using a design thinking approach. *Health Psychology and Behavioral Medicine, 13*(1), Article 2597607. doi:10.1080/21642850.2025.2597607

Weitzman, M. L. (1979). Optimal search for the best alternative. *Econometrica, 47*(3), 641–654. doi:10.2307/1910412

Woodbury, R. F., & Burrow, A. L. (2006). Whither design space? *Artificial Intelligence for Engineering Design, Analysis and Manufacturing, 20*(2), 63–82. doi:10.1017/S0890060406060057